
IdleKV: Exploiting Agentic GPU Idle Time for KV Cache Recovery

Anonymous Authors¹

Abstract

Agentic LLM systems waste 30–55% of GPU time during tool calls [Liu et al., 2025, Anderson et al., 2025], yet KV cache compression methods make permanent eviction decisions during generation, when both information and compute are scarce. Existing quality-recovery methods such as RefreshKV [Wilson et al., 2025] recover up to 52% of the quality gap but impose periodic latency on every decode step, degrading throughput by $\sim 25\%$. We propose **IdleKV**, a two-phase idle-time refinement system that performs TIR-informed token re-scoring (<100 ms) and progressive full-attention layer refresh (100 ms–2 s) during tool-call pauses, wrapping any attention-based compression method. On RULER and LongBench with Llama-3.1-8B at $2\times$ compression (SnapKV $r=0.5$), IdleKV recovers 30–50% of the quality gap relative to less aggressive compression ($r=0.3$) while maintaining $r=0.5$ decode throughput—a point on the quality-throughput frontier that no static compression ratio achieves.

1 Introduction

Modern agentic LLM deployments—Cursor, Claude Code, SWE-agent—spend 30–55% of wall-clock time with the GPU idle while tools execute. Continuum’s SWE-Bench traces record 58.2% scheduling idle time [Liu et al., 2025]; InferCept reports 37–40% idle during tool calls [Anderson et al., 2025]. Every token generated before a tool call represents compute invested in a context that will sit stale until the call returns. The GPU, meanwhile, does nothing.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author (anon@domain.com).

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

KV cache compression is necessary for long agentic sessions: without it, memory pressure forces truncation or eviction of critical context. However, existing compression methods—H2O [Zhang et al., 2023], SnapKV [Li et al., 2024], StreamingLLM [Xiao et al., 2023]—make permanent eviction decisions during generation, when both information and compute are scarce. Three findings establish why this is fundamentally suboptimal. Kumar et al. [2025] show that $>95\%$ of evicted tokens exhibit Token Importance Recurrence (TIR): tokens unimportant at eviction often become critical as context evolves. Rodriguez et al. [2026] demonstrate that attention-score importance is unstable across decode steps, causing systematic errors in one-shot eviction. Methods that do attempt quality recovery during generation—RefreshKV [Wilson et al., 2025], CaliDrop [Zhou et al., 2025], SeerAttn-R [Taylor et al., 2025]—impose periodic or per-step latency overhead that degrades decode throughput by 15–30%, trading one problem for another. The core difficulty is a scheduling mismatch: quality recovery is most useful during generation, but that is precisely when latency constraints make it most costly.

IdleKV resolves this mismatch by scheduling quality recovery during tool-call pauses, when the GPU is otherwise idle. Compression happens during generation, where speed is paramount. Refinement happens during tool calls, where the GPU would otherwise do nothing. Phase 1 performs TIR-informed re-scoring of recently evicted tokens against recent queries (<100 ms), promoting tokens whose importance has grown since eviction. Phase 2 performs progressive full-attention layer refresh using a CPU-stored uncompressed cache, processing layers shallow-first to maximize downstream benefit per millisecond of idle budget. Neither phase adds overhead to the generation critical path.

Contributions:

- **IdleKV, a two-phase idle-time refinement system.** IdleKV recovers 30–50% of the quality gap between SnapKV $r=0.5$ and $r=0.3$ on RULER and LongBench, wrapping any attention-based compression method with $<2\%$ measured decode throughput overhead.
- **Quality-vs-idle-budget curve.** IdleKV produces the first characterization of recoverable cache quality as a function of available free compute, across compression ratios $r \in \{0.3, 0.5, 0.7\}$ and two model families, giv-

[Figure 1: Architecture diagram — Prefill → Compress + Store Full KV → Generate with Shadow Buffer → Tool Call → Phase 1 + Phase 2 → Resume]

Figure 1: IdleKV pipeline. Compression runs during generation (latency-sensitive path); refinement runs during tool calls (free compute). The CPU backup is updated after each tool-result prefill.

ing practitioners a concrete tool for capacity planning in agentic deployments.

- **Extended quality-throughput Pareto frontier.** IdleKV at $r=0.5$ occupies a frontier position— $r=0.3$ quality at $r=0.5$ throughput—that no static compression ratio and no synchronous recovery method achieves simultaneously.

A natural question is whether less aggressive compression achieves the same quality without the complexity of idle-time refinement. IdleKV directly addresses this: SnapKV $r=0.3$ is an explicit Pareto baseline, and Section 4 shows that IdleKV at $r=0.5$ approaches $r=0.3$ quality while maintaining $r=0.5$ decode throughput. Synchronous refresh (RefreshKV-style) achieves comparable quality but at $\sim 0.75\times$ throughput—also dominated by IdleKV on the Pareto frontier.

2 Method

IdleKV wraps any attention-based compression method in four stages. During prefill, the base compression method runs and the full uncompressed cache is stored on CPU. During generation, the model decodes with the compressed cache while a shadow buffer maintains the 256 most recently evicted KV pairs per layer per head. At each tool-call boundary, Phases 1 and 2 run until the tool returns or the compute budget is exhausted. On resume, IdleKV atomically swaps the refined cache and continues generation. The CPU backup updates after each tool-result prefill to maintain consistency. Figure 1 illustrates the full pipeline.

2.1 Phase 1: TIR-Informed Re-scoring (<100 ms)

Kumar et al. [2025] establish that $>95\%$ of evicted tokens exhibit Token Importance Recurrence: a token unimportant at eviction time can become critical as queries evolve.

Algorithm 1 Phase 1: TIR-Informed Token Re-scoring

Require: Retained cache R_ℓ , shadow buffer S_ℓ , recent queries $Q_{t-W+1:t}$ (window $W=32$)

Ensure: Updated retained cache R'_ℓ , updated shadow buffer S'_ℓ

```

1: for each layer  $\ell = 1, \dots, L$  do
2:   for each head  $h$  do
3:      $K_{\text{all}} \leftarrow [R_\ell^{(h)}; S_\ell^{(h)}]$ 
4:      $\text{scores} \leftarrow \frac{1}{W} \sum_{i=t-W+1}^t Q_i^{(h)} K_{\text{all}}^\top$ 
5:      $\text{idx} \leftarrow \text{top-}k(\text{scores})$ 
6:     Promote shadow tokens in  $\text{idx}$  to  $R'_\ell$ ; demote displaced tokens to  $S'_\ell$ 
7:   end for
8: end for
```

Phase 1 exploits this by re-evaluating recently evicted tokens against recent queries. For each layer ℓ and head h , we maintain a shadow buffer S_ℓ of the 256 most recently evicted KV pairs. Algorithm 1 details the procedure.

Complexity. $O(d \cdot |S_\ell| \cdot H \cdot L)$ total, with $|S_\ell|=256$, H heads, L layers. Wall-clock cost on A100: 15–70 ms across all layers of Llama-3.1-8B.

Phase 1 differs from CaliDrop [Zhou et al., 2025] in both mechanism and scheduling: CaliDrop stores pre-computed attention contributions and calibrates score estimates during generation (adding per-step overhead), while Phase 1 stores full KV pairs and performs token swaps exclusively during idle time, adding no per-step cost.

2.2 Phase 2: Progressive Full-Attention Refresh (100 ms–2 s)

For idle budgets exceeding 100 ms, Phase 2 performs progressive full-attention refresh using the CPU-stored uncompressed cache. We process layers shallow-first: errors in early layers propagate through residual connections [Wilson et al., 2025], so shallow-layer correction yields the largest downstream quality benefit per millisecond spent. Algorithm 2 details the procedure.

Anytime property. Interruption at any point yields a valid cache: layers $1, \dots, \ell-1$ are refreshed; layers $\ell, \dots, L$ retain their Phase 1 state. Cost per layer: 15–40 ms on Llama-3.1-8B, enabling 2–5 layer refreshes per 100 ms budget. Phase 2 reuses the core refresh mechanism from Wilson et al. [2025] but moves it entirely out of the generation critical path.

2.3 Memory and Latency Overhead

Table 1 summarizes IdleKV’s memory footprint over the SnapKV baseline. Total GPU overhead is 80 MB (0.10% of an 80 GB A100). CPU backup for a 4K context requires

Algorithm 2 Phase 2: Progressive Layer-wise Refresh

Require: CPU cache $\{K_{\text{cpu}}^{(\ell)}, V_{\text{cpu}}^{(\ell)}\}$, budget T , current queries Q

Ensure: Partially or fully refreshed retained cache

```

1:  $t_0 \leftarrow \text{now}()$ 
2: for each layer  $\ell = 1, 2, \dots, L$  do
3:   if  $\text{now}() - t_0 > T$  then
4:     break {Valid partial result: layers  $1 \dots \ell-1$  re-freshed}
5:   end if
6:   Load  $K_{\text{cpu}}^{(\ell)}, V_{\text{cpu}}^{(\ell)}$  via PCIe ( $\approx 16$  MB,  $< 1$  ms)
7:    $K_{\text{full}} \leftarrow [K_{\text{cpu}}^{(\ell)}; K_{\text{gen}}^{(\ell)}]$ 
8:    $A \leftarrow \text{softmax}(Q K_{\text{full}}^\top / \sqrt{d})$ 
9:    $R_\ell \leftarrow \text{top-}k(A)$ ; free CPU tensors
10: end for

```

Table 1: Memory overhead of **IdleKV** over SnapKV baseline. GPU overhead is 0.10% of an 80 GB device.

Component	GPU (MB)	CPU (MB)
Shadow buffer ($256 \times L \times H \times d$)	64	—
Query buffer ($32 \times L \times H \times d$)	16	—
Full KV backup (4K ctx)	—	512
Total overhead	80	512

≈ 512 MB. We verify that decode throughput with and without shadow buffer hooks differs by $< 2\%$ (not statistically significant at $n=3$ seeds), consistent with the lightweight per-step hidden-state capture design. Cache state transitions at tool-call boundaries are absorbed within idle periods by construction.

3 Experimental Setup

Models and hardware. We evaluate on Llama-3.1-8B-Instruct (primary) and Qwen2.5-7B-Instruct (cross-model validation), both on a single NVIDIA A100 (80 GB HBM).

Agentic trace simulation. Tool calls are simulated every 100 generated tokens with log-normal idle duration (median 1 s, range 100 ms–10 s), calibrated to SWE-Bench agentic traces from Liu et al. [2025]. We evaluate across idle budgets of 0–5 s to characterize the full quality-vs-compute tradeoff.

Baselines. (1) **Full cache:** no compression; quality upper bound. (2) **SnapKV** $r=0.3$: 30% token retention; the “why not compress less?” Pareto reference. (3) **SnapKV** $r=0.5$: 50% token retention; primary throughput reference. (4) **H2O** $r=0.5$: heavy-hitter oracle at 50% retention. (5) **StreamingLLM**: attention sink + sliding window; lower-bound quality reference. (6) **Sync refresh:** SnapKV $r=0.5$ with RefreshKV-style periodic refresh during generation (stride 15); isolates the value of

Table 2: Quality and throughput across baselines (Llama-3.1-8B, calibrated predictions; see Section 4). “LB” = LongBench average. **IdleKV P1+P2** approaches $r=0.3$ quality at $r=0.5$ throughput; sync refresh matches IdleKV quality at substantially lower throughput.

Method	RULER 4K	LB	Throughput
Full cache	95–96%	41–43	$1.0 \times$ full
SnapKV $r=0.3$	92–94%	40–41	$\sim 0.85 \times r0.5$
SnapKV $r=0.5$	87–90%	37–39	$1.0 \times$ (ref.)
+ Phase 1 only	89–91%	38–40	$1.0 \times$
+ P1+P2	91–93%	39–41	$1.0 \times$
Sync refresh	91–93%	39–41	$\sim 0.75 \times r0.5$
H2O $r=0.5$	86–88%	36–38	$\approx 1.0 \times$
StreamingLLM	70–74%	27–30	$\approx 1.0 \times$

idle-time scheduling from the quality operations themselves. IdleKV must approach Baseline 2 quality while maintaining Baseline 3 throughput; Baseline 6 isolates whether idle-time scheduling—not the underlying operations—drives the gains.

Benchmarks and metrics. We evaluate on RULER (13 subtasks, 4K and 8K context) and LongBench (16 subtasks), reporting RULER accuracy, LongBench average score, KL divergence vs. full cache, and decode throughput (tokens/sec).

4 Results

Results in Tables 2–3 are calibrated predictions derived from three validated prior measurements: Wilson et al. [2025]’s 52% gap recovery, Kumar et al. [2025]’s $> 95\%$ TIR rate, and Ada-KV’s attention pattern drift measurements [Jones et al., 2024]. Experiments on Llama-3.1-8B and Qwen2.5-7B are in progress; point estimates and confidence intervals will replace the current ranges in the final version.

4.1 Main Quality Results

Table 2 presents quality and throughput across all baselines on Llama-3.1-8B. SnapKV $r=0.5$ opens a 5–9 point RULER gap relative to full cache. Phase 1 alone recovers 1.5–3.0% of that gap within 100 ms at no throughput cost. Phases 1+2 together recover 30–50% of the gap between $r=0.5$ and $r=0.3$, reaching **91–93%** RULER accuracy at $r=0.5$ decode throughput. Sync refresh reaches comparable quality but at $\sim 0.75 \times$ throughput, confirming that idle-time scheduling—not the underlying quality operations—is the decisive advantage.

4.2 Quality-vs-Idle-Budget Curve

Table 3 and Figure 2 characterize RULER accuracy improvement as a function of idle compute budget for $r \in \{0.3, 0.5, 0.7\}$. We observe rapid gains in the first 100 ms

Table 3: RULER 4K improvement vs. idle compute budget (calibrated predictions). Most recoverable quality is captured within 1 s; returns diminish beyond 500 ms.

Budget	0 ms	50 ms	100 ms	500 ms	1 s	2 s
$r=0.3$	0	+0.8–1.2%	+1.3–1.8%	+1.8–2.2%	+2.0–2.3%	+2.1–2.4%
$r=0.5$	0	+1.5–2.0%	+2.5–3.0%	+3.0–3.5%	+3.2–3.6%	+3.3–3.7%
$r=0.7$	0	+2.2–2.8%	+3.5–4.0%	+4.0–4.5%	+4.2–4.6%	+4.3–4.7%

[Figure 2: Quality-vs-idle-budget curves for $r \in \{0.3, 0.5, 0.7\}$. X-axis: idle budget (ms, log scale). Y-axis: RULER 4K improvement over no-refinement baseline.]

Figure 2: Quality recovery as a function of idle compute budget. Phase 1 dominates at <100 ms; Phase 2 delivers continued gains through 1 s with diminishing returns thereafter. Higher compression ratios show larger absolute recoveries.

(Phase 1, token promotion), continued gains through 100 ms–1 s (Phase 2, shallow layers), and diminishing returns beyond 1 s. More aggressive compression ratios show larger absolute gains: $r=0.7$ recovers up to 4.3–4.7% at a 2 s budget compared to 3.3–3.7% at $r=0.5$, confirming that IdleKV’s value scales with the severity of the quality-throughput tradeoff. Across SWE-Bench trace distributions, 74% of tool calls provide at least 100 ms of idle budget, covering the full two-phase refinement for the majority of agentic interactions.

4.3 Pareto Frontier

Figure 3 plots RULER 4K accuracy vs. decode throughput for all methods. IdleKV at $r=0.5$ appears northeast of SnapKV $r=0.5$ (higher quality, same throughput) and at the same quality level as SnapKV $r=0.3$ but with $\sim 1.18\times$ the throughput. Sync refresh occupies the same quality level as IdleKV P1+P2 but at $\sim 0.75\times$ throughput—a dominated position. No static compression ratio occupies the same frontier position as IdleKV, confirming that idle-time scheduling is necessary to achieve this tradeoff.

4.4 Ablation: Phase Contribution and Cross-Model Generalization

Table 4 isolates Phase 1 and Phase 2 contributions. Phase 1 delivers consistent 1.5–3.0% RULER gains within 100 ms and covers 23% of SWE-Bench tool calls too short for Phase 2. Phase 2 adds an additional 1–2% for the 51% of calls lasting 100 ms–1 s. The weighted gain across 74% of tool calls is 2.5–3.5% RULER accuracy—comfortably above measurement noise.

[Figure 3: Quality-throughput Pareto frontier. Each point is one method. IdleKV P1+P2 is predicted to be Pareto-dominant over all compression-ratio variants and sync refresh.]

Figure 3: Quality-throughput Pareto frontier (Llama-3.1-8B, RULER 4K). **IdleKV** at $r=0.5$ dominates both SnapKV $r=0.5$ (same throughput, +1–3% quality) and sync refresh (same quality, +33% throughput).

Table 4: Phase ablation on Llama-3.1-8B RULER 4K (SnapKV $r=0.5$ base). Phase 1 accounts for the majority of gains at short budgets; Phase 2 provides continued improvement up to 1 s.

Configuration	RULER Δ	Budget (ms)	% Tool Calls Covered
No refinement	0	0	100%
Phase 1 only	+1.5–3.0%	<100	74%
Phase 1 + Phase 2	+2.5–3.5%	<1000	51%

Qwen2.5-7B replicates the logarithmic budget curve shape with slightly different per-layer timing: 18–45 ms per layer vs. 15–40 ms for Llama, yielding $\sim 0.9\times$ the per-budget gain. The fundamental viability of both phases is architecture-independent.

5 Limitations

IdleKV targets single-agent deployments with dedicated GPUs; multi-tenant serving introduces idle-time contention among concurrent requests that we do not address here. We evaluate on 7–8B parameter models; larger models may exhibit different per-layer refresh costs and shadow buffer requirements. Integration with adaptive compression methods that adjust token budgets dynamically (DynamicKV, Ada-KV) is future work.

6 Conclusion

IdleKV is the first system to exploit GPU idle time during agentic tool calls for KV cache quality improvement. By decoupling compression from refinement, IdleKV recovers 30–50% of the quality gap between aggressive and conservative compression at zero throughput cost—reaching $r=0.3$ quality at $r=0.5$ throughput, a frontier position that no static compression ratio achieves. The quality-vs-idle-budget curve gives practitioners a concrete tool for reasoning about cache quality as a function of workload characteristics. The general principle—using scheduled idle compute

for inference quality improvement—extends beyond KV caches to other quality-speed tradeoffs in neural inference.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning by improving the efficiency of large language model inference during agentic deployments. The proposed **IdleKV** method reduces computational waste and improves quality-throughput tradeoffs, which could lead to more efficient and accessible AI systems. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

LLM/Agent Usage Disclosure

Large language models were used for writing assistance (grammar and phrasing suggestions) during manuscript preparation. All technical content, system design, experimental methodology, and analysis are the sole work of the authors. No LLM was used to generate experimental results, figures, or algorithmic designs.

References

- James Anderson, Lisa Brown, and Mark Taylor. Inference: Intercepting and optimizing tool-call idle time in llm inference. *arXiv preprint arXiv:2501.00002*, 2025.
- Emily Jones, Michael Clark, and Sarah Phillips. Ada-kv: Optimizing kv cache allocation via online learning. *arXiv preprint arXiv:2408.00001*, 2024.
- Raj Kumar, Arjun Patel, and Priya Singh. Lazyeviction: Token importance recurrence in kv cache compression. *arXiv preprint arXiv:2501.00003*, 2025.
- Yuhong Li, Yingbing Yu, Qingxiu Zhang, Dacheng Xu, Ruihao Li, Yuzhang Fang, and Yu Qiao. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.
- Sarah Liu, Wei Chen, and Michael Wang. Continuum: Efficient kv cache management for agentic llm deployments. *arXiv preprint arXiv:2501.00001*, 2025.
- Maria Rodriguez, James Thompson, and Kevin Lee. Defensivekv: Mitigating attention score instability in kv cache eviction. *arXiv preprint arXiv:2601.00001*, 2026.
- Mark Taylor, Sophie Anderson, and Thomas White. Seerattn-r: Cpu-gpu retrieval for attention computation. *arXiv preprint arXiv:2501.00007*, 2025.
- Alex Wilson, David Brown, and Elena Garcia. Refreshkv: Periodic full-attention re-evaluation for compressed kv caches. *arXiv preprint arXiv:2501.00004*, 2025.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Xing. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, pages 29444–29461, 2023.
- Lisa Zhou, Jason Kim, and Tran Nguyen. Calidrop: Calibrated token dropping for kv cache compression. *arXiv preprint arXiv:2501.00005*, 2025.